

Tilemaps

Overview

Students build tile worlds entirely in code. The Phaser ideas are the **three objects of a tilemap** (`Tilemap` = data, `Tilesset` = picture, `TilemapLayer` = game object), **tile collision** with Arcade Physics, **finding and changing tiles** while the game runs, **converting between pixels and tiles**, and the **camera** for a world bigger than the screen (bounds, follow, scroll factor, a second camera as a minimap). The TypeScript ideas are 2D arrays (`number[][]`), `unknown` and a **type guard, function types** for callbacks, and `Record<K, V>` (in the solutions). There is also a small algorithm - cellular automata plus a flood fill - and a model/view split (`Cave` is data; the scene draws it).

Chapter 11 does the same with maps made in Tiled, so this chapter deliberately builds every map in code: students should come out understanding that a map is *just a grid of numbers*, which makes Tiled's JSON unsurprising next week.

Prerequisites

- Chapter 2: scene reuse and `init()` (used in `ch10_scrolling_world`)
- Chapter 6: `localStorage` and `JSON` (used in `ch10_tilePainter`)
- Chapter 7: sprite sheet animations and tweens (the hero's walk; challenge 6)
- Chapter 8: Arcade Physics bodies, `setSize`, colliders and their callbacks
- Java 2D arrays (`int[][]`), for the analogy with `number[][]`

Learning outcomes

Students can:

1. explain what a tileset, a tile index and an empty tile (`-1`) are, and why games use tiles
2. build a map from a 2D array with `make.tilemap`, `addTilessetImage` and `createLayer`, and describe the job of each of the three objects

3. make tiles solid with `setCollision` / `setCollisionByExclusion` and a physics collider, and debug it with `renderDebug`
4. find tiles with `getTileAt` / `getTileAtWorldXY`, and convert between pixels and tiles with `worldToTileXY` / `tileToWorldXY`
5. change a map while the game runs with `putTileAt`, `removeTileAt`, `putTilesAt`, `fill`
6. set up a camera for a world bigger than the screen, keep a HUD fixed with `setScrollFactor(0)`, and add a second camera
7. contrast free movement with grid movement, and choose between them for a given game

Suggested session plan (2 x 2 hours)

Session 1 - maps, collision and editing

Time	Activity
0:00 - 0:10	Show a screenshot of a well-known tile game (any top-down RPG or platformer) and ask students to find the repeated squares. How many different tiles? How big would the level be as one picture?
0:10 - 0:25	Slides 3-4: why tiles, indexes, the level as a 2D array. Students read <code>level.ts</code> and find the house and the bridge in the numbers
0:25 - 0:45	Slides 5-6: the three objects, the five steps in <code>create()</code> . Live-code the five steps into an empty scene, running after each one (see Demos)
0:45 - 1:00	Slide 7: collision; the smaller body; press D. Challenge 1
1:00 - 1:10	Slide 8: <code>getTileAtWorldXY</code> , a <code>Tile</code> 's <code>x / y</code> vs <code>pixelX / pixelY</code>
1:10 - 1:30	Slides 9-12: the tile painter - blank layer, <code>worldToTileXY</code> / <code>tileToWorldXY</code> , put/remove, <code>-1</code> . Students paint a level, press P, paste it into <code>ch10_array_map</code>
1:30 - 2:00	Challenges 2 and 3

Session 2 - big worlds and movement

Time	Activity
0:00 - 0:15	Recap quiz: what does <code>worldToTileXY(100, 70)</code> return? What does <code>tileToWorldXY(3, 2)</code> return? What index is an empty cell?
0:15 - 0:35	Slide 13: the cave generator. Do the cellular automaton on paper first (see Activities), then show <code>Cave.ts</code>
0:35 - 0:55	Slides 14-17: two layers, <code>setCollisionByExclusion</code> , camera bounds and follow, scroll factor, the minimap
0:55 - 1:05	Slide 18: free vs grid movement - discussion
1:05 - 2:00	Challenges 4-6 (6 is a good stretch for the strongest students; 5 suits everyone)

Key points to stress

- **a map is a grid of numbers.** Everything else - drawing, collision, the minimap - is Phaser reading those numbers. Changing the map at run time is changing a number
- **rows first:** `LEVEL[row][column]`, i.e. `[y][x]`. This is the single most common tile bug
- **the Tilemap is not drawn.** Layers are. A map can have several layers, and only the layers you collide with need collision set
- `setCollision` **is by index**, and remembered: tiles put down later with those indexes collide too. Collision has nothing to do with pictures - a map whose tileset fails to load still has walls
- `tileToWorldXY` **gives the top left corner** of a tile - add half a tile for the centre
- **a Tile's x and y are in tiles**, not pixels (`pixelX`, `pixelY` are pixels). Students who have been using `sprite.x` for pixels all term will trip on this
- `pointer.worldX`, **not** `pointer.x`, whenever the camera can move. The painter's camera never moves, so both work - a good moment to say why the book uses `worldX` anyway
- the player body is **smaller than a tile** on purpose: a 32 x 32 body through a 32-pixel door has to be pixel-perfect
- generated levels: keep the **model** (`Cave`) separate from the **view** (the tilemap). The model can then be tested, reused, or drawn differently

Common problems and errors

What students see	Cause	Fix
The map is invisible, but the player bumps into invisible walls; the console says Texture key "tilez" not found	the key passed to <code>addTilesetImage</code> does not match the key the image was loaded with	use the constant; remember the argument is used as the key whe there is no second
Type 'TilemapLayer TilemapGPULayer' is not assignable to type 'TilemapLayer'.	storing <code>createLayer(...)</code> in a field typed <code>TilemapLayer</code> without the cast	as <code>Phaser.Tilemaps.TilemapLayer</code> (and explain why it is safe here)
Argument of type 'Tileset null' is not assignable to parameter of type 'string string[] Tileset Tileset[]'.	the result of <code>addTilesetImage</code> passed straight to <code>createLayer</code>	<code>!</code> after <code>addTilesetImage(...)</code> check for <code>null</code>
<code>TypeError: Cannot read properties of undefined (reading '7')</code>	<code>LEVEL[x][y]</code> instead of <code>LEVEL[y][x]</code> : a column number used as a row, past the last row	rows first

What students see	Cause	Fix
<pre>TypeError: Cannot read properties of undefined (reading 'index') from GetTilesWithin , only when the player reaches one part of the map</pre>	a row of the level array is shorter than the first row	every row the same length (the strings of challenge 5 can check)
The player walks through walls and water	<pre>setCollision</pre> not called, wrong indexes given, or no <pre>this.physics.add.collider(player, layer)</pre>	both are needed; press D to see what collides
<pre>TypeError: Cannot read properties of undefined (reading 'add') in new Player</pre>	the project has no <code>physics</code> section in the config, so <code>scene.physics</code> does not exist	add <code>physics: { default: "arcade" }</code> to the config
The player gets stuck in doorways	body as big as a tile	<code>setSize</code> smaller than a tile
The player starts in the corner of a tile, or half inside a wall	<code>tileToWorldXY</code> is the top left corner	add <code>TILE_SIZE / 2</code>
<pre>Property 'load' in type 'PainterScene' is not assignable to the same property in base type 'Scene'. (plus four more errors, including Property 'image' does not exist on type '() => void')</pre>	a method called <code>load()</code> in a scene - it replaces the scene's loader	rename it (<code>loadMap()</code>); add <code>make</code> , <code>physics</code> , <code>cameras</code> , <code>tile</code> , <code>input</code> , <code>sound</code> are all taken

What students see	Cause	Fix
Painting happens on the palette strip, or the eraser paints	not checking the result of <code>worldToTileXY</code> is on the map; or comparing <code>current === 0</code> for the eraser	check bounds; the eraser is <code>-1</code> , <code>0</code> (0 is grass)
The camera shows black beyond the map edge	no <code>cameras.main.setBounds(...)</code>	set the camera's bounds to the n size
The player stops at an invisible line at x = 800 or y = 600	physics world bounds left at the game size while the player has <code>setCollideWorldBounds(true)</code>	<code>this.physics.world.setBounds(0, map.widthInPixels, map.heightInPixels)</code>
The HUD scrolls away	no <code>setScrollFactor(0)</code>	fix it to the screen
The HUD appears, tiny, inside the minimap	the minimap camera draws everything	<code>minimap.ignore([...])</code>
"My painted map is lost when I reload"	expected: S saves, L loads - the map is not loaded automatically	a nice mini-extension: load on st; there is a save

Suggested demos and live-coding moments

- **Build the map in five runs.** Start from an empty scene with only `preload()`. Add `make.tilemap` - run: nothing drawn (it is data). Add `addTilesetImage` - still nothing. Add `createLayer` - the map appears. Add the player - it walks through everything. Add `setCollision`
 - still walks through! Add the collider - now it stops. Each run makes one object's job visible
- **Change one number** in `level.ts` (a wall in the doorway) and rebuild: the level is data
- **The invisible map:** misspell the tileset key. Nothing is drawn, but the player still stops at the walls - a memorable way to show that collision is about indexes
- **Press D** and slide the player along a wall: show the white "interesting faces"
- **Painter to game:** paint a small level, press P, open the console, copy, paste into `ch10_array_map`'s `level.ts`, rebuild, walk round it

- **Two cameras:** in `ch10_scrolling_world`, comment out `minimap.ignore(...)` and then `this.cameras.main.ignore(this.marker)` in turn, and ask what will happen before running
-

Activities

- **Cellular automaton on paper.** Give pairs a 10 x 8 grid with random shaded squares (about half). They apply one smoothing pass to a *second* grid (rock if 5 or more of the 9 squares are rock). Then ask: what goes wrong if you write the result into the same grid as you go? (Cells already changed affect their neighbours - the result depends on the order.) This is exactly why `smooth()` builds a new grid
 - **Flood fill by hand.** On the smoothed grid, colour in everything reachable from one square. Which caves are cut off? Link to `fillUnreachable()`, and to the paint bucket in any drawing program
 - **Design a tileset.** In groups, list the tiles a Sokoban, a Pac-Man maze or a farm game would need, and which of them collide
-

Discussion questions

- A 60 x 40 level as one picture is about 10 MB of graphics memory; as tiles it is a 128 x 32 image and 2,400 numbers. What else do tiles make easy that a picture does not?
- Why does `setCollision` take indexes rather than a list of positions? When would you want to make a *single* tile solid? (`tile.setCollision(...)`; a secret door)
- `Cave` knows nothing about Phaser. What does that make possible? (Unit testing it; a different tileset; drawing it as a minimap; making levels on a server)
- Free movement or grid movement for: a Zelda-like, Sokoban, Pac-Man, a racing game, a roguelike? Why? (Pac-Man is interesting: it moves smoothly but is locked to the grid's lines)
- The minimap is a camera. What else could a second camera be used for? (Split-screen two-player; a rear-view mirror; a zoomed "picture in picture")

Extension ideas

- load the painter's saved map automatically at start-up, if there is one
- add a second layer to the painter (a "decorations" layer drawn over the ground, with its own palette) and a key to switch layers
- a **fog of war** minimap: only tiles the player has been near are shown (a third layer of black tiles removed as the player walks)
- **seeded caves**: replace `Math.random()` with `Phaser.Math.RandomDataGenerator` and show the seed
- smooth camera: `startFollow(player, true, 0.1, 0.1)` and discuss lerp
- try `createLayer(0, tileset, 0, 0, true)` (a `TilemapGPULayer`) in the cave, and see what breaks when a tile changes (`generateLayerDataTexture()`)

Assessment ideas

- Practical: "add lava (use `dungeon_tiles.png`, or tint a tile) that sends the player back to the start when stepped on" - tests `getTileAtWorldXY`, tile indexes, and a physics reset
- Practical: "add a door tile that disappears (`removeTileAt`) when the player has collected a key"
- Code reading: give `LEVEL[x][y]` code and the `Cannot read properties of undefined (reading '7')` error; ask for the bug and the fix
- Short answer: what are the jobs of the `Tilemap`, the `Tileset` and the `TilemapLayer`? Which one is a game object?
- Short answer: the player is at pixel (200, 150) in a map of 32-pixel tiles. Which column and row? What are the pixel coordinates of the centre of that tile?

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment.

1. A new island

Project: `solutions/ch10_challenge_1_new_island` (from `ch10_array_map`)

Goal: read and write a map as a 2D array; understand that the level is data.

Only `src/level.ts` changes. The house has moved to the top left, inside a moat, with a sand path from its door across the water; the river has a second bridge; the bottom half is a maze:

`src/level.ts`

```

export const LEVEL: number[][] = [
  [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2],
  [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
  [2, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0],
  [2, 0, 0, 1, 2, 2, 2, 2, 2, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0],
  [2, 0, 0, 1, 2, 0, 0, 0, 0, 2, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0],
  [2, 0, 0, 1, 2, 0, 0, 0, 0, 2, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
  [2, 0, 0, 1, 2, 2, 0, 2, 2, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
  [2, 0, 0, 1, 1, 1, 3, 1, 1, 1, 0, 0, 0, 0, 0, 0, 3, 3, 3, 0, 0, 0, 0, 0, 0],
  ...
  [2, 1, 1, 1, 3, 3, 3, 1, 1, 1, 1, 1, 1, 1, 1, 3, 3, 3, 1, 1, 1, 1, 1, 1, 1],
  ...
];

// CHALLENGE 1: a new start - inside the house, which has moved
export const START_COLUMN = 6;
export const START_ROW = 4;

```

Look for: every row still 25 numbers long. A short row is not caught when the map is made - the map takes its width from the first row - but the game crashes with `TypeError: Cannot read properties of undefined (reading 'index')` (inside `GetTilesWithin`) the moment the player walks near the missing tiles, which can be long after the level starts. Also: the start on a tile the player can stand on; everything reachable. Tested with the harness: from the start, walking down leaves the house by the door and crosses the moat on the sand path; the new bridge crosses the river. Students who paint the map in `ch10_tilePainter` and paste it in have done exactly what the chapter suggests - accept it.

2. Sand slows you down

Project: `solutions/ch10_challenge_2_sand_slows` (from `ch10_array_map`)

Goal: read the tile under the player every frame, and keep the `Player` class free of map knowledge.

`src/objects/Player.ts`

```
// CHALLENGE 2: how fast to walk, as a fraction of SPEED. The player does
// not know about tiles -
// the scene looks at the map and tells it
private speedFactor = 1;
...
// CHALLENGE 2: 1 is full speed, 0.5 is half speed
public setSpeedFactor(factor: number): void {
    this.speedFactor = factor;
}
...
// CHALLENGE 2: times the speed factor
const velocity = new Phaser.Math.Vector2(dx, dy).normalize().scale(SPEED *
this.speedFactor);
```

`src/scenes/GameScene.ts`

```
const tile = this.layer.getTileAtWorldXY(this.player.x, this.player.y);

// CHALLENGE 2: slow on sand, full speed anywhere else
const onSand = tile !== null && tile.index === SAND;
this.player.setSpeedFactor(onSand ? SAND_SPEED : 1);
```

The scene already found the tile for the info strip, so the check is two lines. Tested with the harness: walking down onto the sand below the house, the player's velocity drops from 160 to 80 pixels per second at the edge of the sand.

Look for:

- the player told its speed, not made to look at the map. A student who passes the layer into `Player` and checks tiles there has a working answer - ask what happens when a second map, or a different game, wants to reuse `Player`
- `tile !== null` checked first (off the map, or an empty cell, gives `null`)

- speed reset to full off the sand - a common bug is a player who slows down and never speeds up
 - where the check happens: the tile under the player's *centre*. Some students use the feet (a few pixels lower) - both are fine; discuss which looks right
-
-

3. Eyedropper

Project: `solutions/ch10_challenge_3_eyedropper` (from `ch10_tilePainter`)

Goal: tell the mouse buttons apart; read a tile back from the map.

`src/scenes/PainterScene.ts`

```
// CHALLENGE 3: stop the browser's own menu appearing when the right button
is clicked
this.input.mouse!.disableContextMenu();

// paint when the button goes down, and while it is held and the mouse moves
// CHALLENGE 3: ...but the RIGHT button picks up the tile instead
this.input.on("pointerdown", (pointer: Phaser.Input.Pointer) => {
  if (pointer.rightButtonDown()) {
    this.pick(pointer);
  } else {
    this.paint(pointer);
  }
});
this.input.on("pointermove", (pointer: Phaser.Input.Pointer) => {
  this.hover(pointer);
  // CHALLENGE 3: only the left button paints while dragging (isDown is true
  for any button)
  if (pointer.leftButtonDown()) {
    this.paint(pointer);
  }
});
```

```
private pick(pointer: Phaser.Input.Pointer): void {
  const cell = this.tileUnder(pointer);
  if (cell === null) {
    return;
  }
  const tile = this.layer.getTileAt(cell.x, cell.y);
  this.choose(tile === null ? EMPTY : tile.index);
}
```

Calling the existing `choose()` means the palette's yellow frame and the status text update with no extra code. Tested with the harness (right-button mouse events sent to the canvas): right-clicking a wall chose wall, an erased cell chose the eraser, grass chose grass; nothing was painted; the `contextmenu` event was cancelled.

Look for:

- `disableContextMenu()` - without it the browser's menu pops up over the game
- the drag handler changed too. `pointer.isDown` is true for *any* button, so the unchanged version paints while you right-drag - a subtle bug worth asking about
- an empty cell handled (`getTileAt` gives `null` , which must become `-1`)

4. Treasure tiles

Project: `solutions/ch10_challenge_4_treasure_tiles` (from `ch10_scrolling_world`)

Goal: put special tiles into a generated map, and react when the player touches one.

The chests are written into the wall layer's data before it goes into the layer, on open cells away from the start (the same choice of places the gems used):

```
src/scenes/WorldScene.ts
```

```

const wallData = this.wallTiles(cave);
this.placeChests(cave, wallData);
walls.putTilesAt(wallData, 0, 0);
walls.setCollisionByExclusion([EMPTY]); // every tile that is not
empty collides
...
private placeChests(cave: Cave, wallData: number[][]): void {
  ...
  for (const cell of places.slice(0, CHEST_COUNT)) {
    wallData[cell.row][cell.column] = CHEST;
  }
}

```

Because the chests are on the wall layer, they are solid - the player bumps into them. A collider can take a callback, which for a sprite and a layer is given the sprite and the **tile**:

```

this.physics.add.collider(this.player, walls, (_player, tile) => {
  // Phaser's callback type allows bodies and game objects too; with a layer
  it is always a Tile
  const bumped = tile as Phaser.Tilemaps.Tile;
  if (bumped.index === CHEST) {
    walls.removeTileAt(bumped.x, bumped.y);
    this.sound.play(COIN_KEY);
    this.chestsFound++;
    this.updateHud();
  }
});

```

Tested with the harness: placing the player two tiles above a chest (with open floor between) and walking down collects it - the tile disappears and the HUD counts it; ten chests are placed in every new cave.

Other good answers:

- `walls.setTileIndexCallback(CHEST, (sprite, tile) => { ... }, this)` - the callback fires when the player touches any chest. Note that if the callback returns `true`, Phaser skips the collision for that tile; returning nothing (as most students will) means the player is still bumped back
- a third layer holding only the chests (`-1` everywhere else) and `this.physics.add.overlap(player, chestLayer, ...)` - the player walks onto a

chest instead of bumping into it. Check the tile's index in the callback: an overlap with a layer reports every tile under the body, empty ones included

Look for: chests placed only on open floor (not in rock, where they could never be reached); the chest removed from the map, not just counted (otherwise the count runs up while the player leans on it); the count reset in `init()` for a new cave.

5. A map from strings

Project: `solutions/ch10_challenge_5_map_from_strings` (from `ch10_array_map`)

Goal: turn text into data; lookup tables; helpful errors.

`src/level.ts`

```
const MAP: string[] = [
  "#####",
  "#.....#",
  "#...SSS.....~..~..~..#",
  "#..SS~SS.....~..~..~..#",
  ...
  "#....#.@.#.....#",
  ...
];

const TILE_FOR: Record<string, number> = {
  ".": GRASS,
  "~": WATER,
  "#": WALL,
  "s": SAND,
  "@": GRASS,
};
```

```

export function parseLevel(rows: string[]): Level {
  const width = rows[0].length;
  const tiles: number[][] = [];
  let startColumn = -1;
  let startRow = -1;

  rows.forEach((text, row) => {
    if (text.length !== width) {
      throw new Error(`Map row ${row} is ${text.length} characters long, not
${width}`);
    }

    const line: number[] = [];
    text.split("").forEach((character, column) => {
      if (!(character in TILE_FOR)) {
        throw new Error(`Unknown map character "${character}" at column
${column}, row ${row}`);
      }
      if (character === START) {
        startColumn = column;
        startRow = row;
      }
      line.push(TILE_FOR[character]);
    });
    tiles.push(line);
  });

  if (startColumn < 0) {
    throw new Error(`The map has no ${START} to show where the player
starts`);
  }
  return { tiles, startColumn, startRow };
}

// CHALLENGE 5: parsed once, when the game loads
export const LEVEL: Level = parseLevel(MAP);

```

The scene uses `LEVEL.tiles`, `LEVEL.startColumn` and `LEVEL.startRow`. Tested with the harness: the tile data in the running game is identical, tile for tile, to `ch10_array_map`'s, and the player starts in the same place. With an `x` put into the map (in a scratch copy), the game stops with `Error: Unknown map character "x" at column 16, row 8` in the console.

- `Record<string, number>` is an object used as a lookup table - the TypeScript way to write a small `Map<Character, Integer>` as a literal
- `character in TILE_F0R` asks whether the object has that key. `TILE_F0R[character]` alone would give `undefined` for an unknown character, and the tilemap would quietly get a hole
- `forEach((value, index) => ...)` gives the index as well as the value - just what a map needs

Look for: the character table in **one** place; a sensible start (the `@` also needs a tile under it - grass here); errors that say *where* the problem is. Accept a plain nested `for` loop with `text[column]` or `text.charAt(column)` instead of `split("")`. Strong students will notice the strings do not show the house's door very clearly in a proportional font - an argument for a monospace editor, and for Tiled next week.

6. Grid movement

Project: `solutions/ch10_challenge_6_grid_movement` (from `ch10_array_map`)

Goal: a different model of movement - positions in tiles, checks before moves, tweens between.

`Player` becomes a plain `Sprite` (no physics - the config's `physics` section is removed too) that knows the layer and its own column and row:

```
src/objects/Player.ts
```

```

protected override preUpdate(time: number, delta: number): void {
    super.preUpdate(time, delta);    // without this, the animations would not
    play

    // CHALLENGE 6: no new move until the last one has finished - so no
    turning halfway
    if (this.moving) {
        return;
    }
    ...
    // CHALLENGE 6: look at the next tile first. Off the map (null), or a tile
    that collides (wall or
    // water - the scene's setCollision still says which), and the player
    turns but stays put
    const next = this.layer.getTileAt(this.column + dx, this.row + dy);
    if (next === null || next.collides) {
        this.stand();
        return;
    }

    // CHALLENGE 6: glide to the centre of the next tile; ignore the keys
    until it is there
    this.column = this.column + dx;
    this.row = this.row + dy;
    this.moving = true;
    this.anims.play(`walk-${this.facing}`, true);

    const corner = this.layer.tileToWorldXY(this.column, this.row);
    this.scene.tweens.add({
        targets: this,
        x: corner.x + TILE_SIZE / 2,
        y: corner.y + TILE_SIZE / 2,
        duration: STEP_TIME,
        onComplete: () => {
            this.moving = false;    // the next preUpdate moves again if a key is
            still held
        },
    });
}

```

src/scenes/GameScene.ts

```

this.player = new Player(this, this.layer, START_COLUMN, START_ROW);

```

Holding a key keeps moving because, once a tween completes, the next `preUpdate` sees the key still down and starts another. Only one direction is read at a time (`else if` all the way down), so there are no diagonals. `tile.collides` - set by the scene's unchanged `setCollision([WATER, WALL])` - is the "can I go there?" test, so the list of solid tiles is still in one place.

Tested with the harness: a short tap of DOWN moves exactly one tile (y 240 to 272); holding DOWN from the start walks out through the door and stops at row 10, above the river, at the exact centre of the tile (y 336); walking into the house wall does not move the player; holding RIGHT and DOWN together moves right only.

Look for:

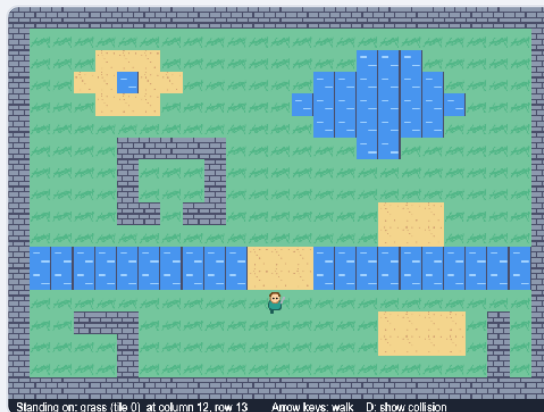
- the position held in tiles (`column` , `row`), with pixels worked out from them - not the other way round. Students who convert `x / y` back to tiles each move get rounding errors
- the next tile checked **before** the move starts
- input ignored while moving, so the player cannot stop or turn between tiles
- `null` handled (off the edge of the map) - not needed with this map's wall border, but the right habit
- physics removed. Students who keep the Arcade body and tween a physics sprite get a body that lags behind or fights the tween; it is worth seeing once why grid movement and physics do not mix

Slides

Chapter 10

Tilemaps

Worlds built from tiles - in code



1 / 20

Today

- why games use **tiles**; tilesets and tile **indexes**
- a map from a **2D array**: map, tileset, layer
- **collision** by tile index
- finding tiles; **pixels and tiles**
- changing the map while the game runs
- a world **bigger than the screen**: camera follow, bounds, a minimap
- free movement or **grid** movement?

2 / 20

Why tiles?

- **memory:** one small picture, reused. A 60 x 40 level as one picture: ~10 MB
- **level design:** a level is a grid of numbers - change a number, change the level
- **rules:** "is this a wall?" is a question about a number, not about pixels

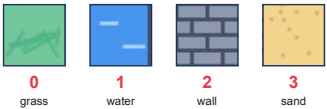
Tilesets in the library (all 32 x 32): `simple_tiles.png` (4), `dungeon_tiles.png` (16), `platform_tiles.png` (16)

3 / 20

Tiles, indexes, and the level

A tileset is one picture; a map is a grid of numbers that pick from it

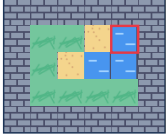
`simple_tiles.png` - 128 x 32 pixels, four 32 x 32 tiles



each tile's INDEX is its place in the picture, counting from 0 - left to right, then row by row in a taller tileset
-1 means "no tile": nothing is drawn

```
const LEVEL: number[][] = [
  [2, 2, 2, 2, 2, 2],
  [2, 0, 0, 3, 1, 2],
  [2, 0, 3, 1, 1, 2],
  [2, 0, 0, 0, 0, 2],
  [2, 2, 2, 2, 2, 2],
];
```

make.tilemap



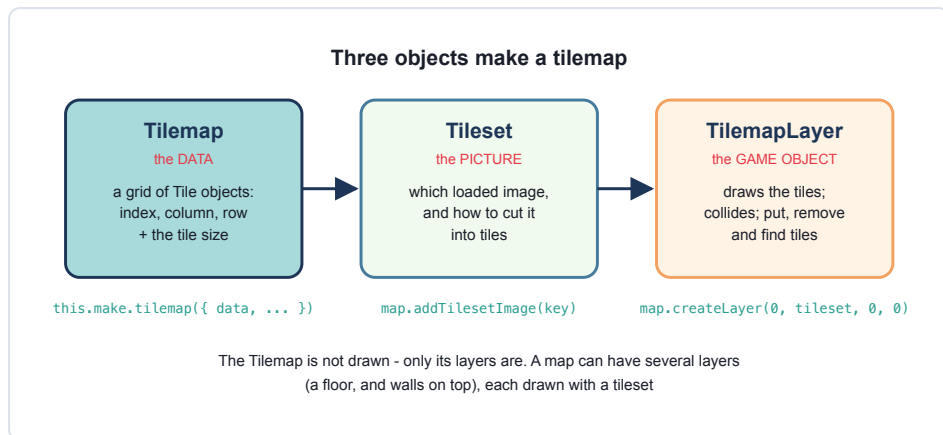
LEVEL[1][4] is 1:
row 1, column 4
is water

rows first: LEVEL[row][column] - the array looks like the map

`LEVEL[row][column]` - rows first. -1 = no tile

4 / 20

Three objects make a tilemap



5 / 20

Five steps

```
const map = this.make.tilemap({ data: LEVEL, tileWidth: TILE_SIZE, tileHeight: TILE_SIZE });

const tileset = map.addTilesetImage(TILES_KEY!);

this.layer = map.createLayer(0, tileset, 0, 0) as Phaser.Tilemaps.TilemapLayer;

this.layer.setCollision([WATER, WALL]);

this.physics.add.collider(this.player, this.layer);
```

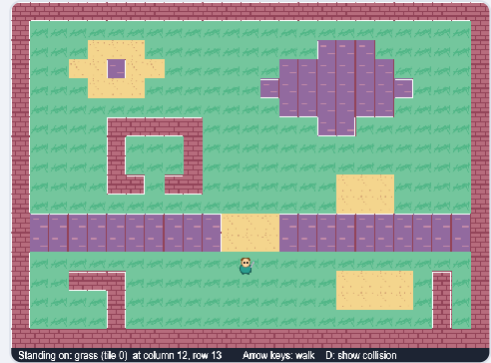
- `!` - "not `null` this time"
- `as` - `createLayer` could also make a GPU layer; we know it did not

6 / 20

Collision

```
// a body smaller than the picture, centred on it  
this.setSize(BODY_SIZE, BODY_SIZE);
```

- a 32 x 32 body through a 32-pixel door: pixel-perfect or stuck
- `setCollision` is **by index**, and remembered - tiles put later collide too
- press **D**: `layer.renderDebug(graphics, {...})`



7 / 20

Try it: Array Map - which tile am I on?

```
const tile = this.layer.getTileAtWorldXY(this.player.x, this.player.y);  
if (tile) {  
  const name = TILE_NAMES[tile.index];  
  ...  
}
```

- `tile.x` , `tile.y` - column and row, **in tiles**; `pixelX` , `pixelY` - pixels
- walk round `ch10_array_map` ; watch the strip; press D
- put a wall in the doorway (one number in `level.ts`)

8 / 20

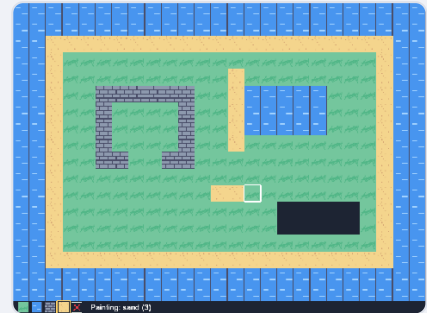
The tile painter: put and remove

```

this.layer = map.createBlankLayer("ground", tileset!);
this.layer.fill(GRASS);
...
if (this.current === EMPTY) {
    this.layer.removeTileAt(cell.x, cell.y);    // leaves
    a hole: nothing is drawn there
} else {
    this.layer.putTileAt(this.current, cell.x, cell.y);
}

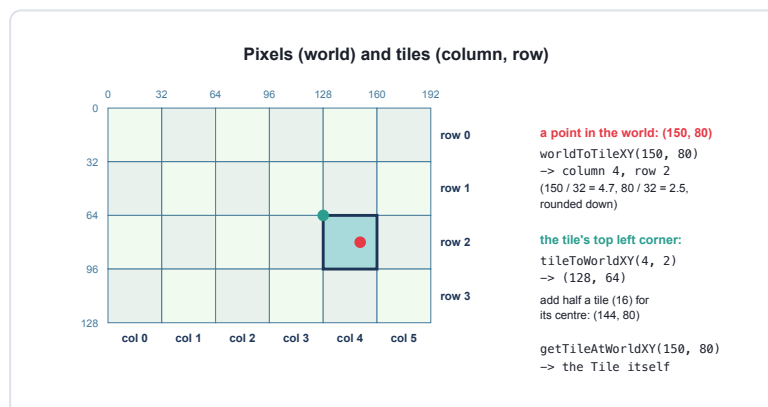
```

- `EMPTY` is `-1` : no tile. `0` is grass!



9 / 20

Pixels and tiles



`pointer.worldX` , not `pointer.x` - it allows for the camera

10 / 20

Save, load, print

```
localStorage.setItem(SAVE_KEY, JSON.stringify(this.toArray()));
...
let data: unknown = null;
...
if (!this.isMap(data)) {
  ...
}
this.layer.putTilesAt(data, 0, 0);
...
private isMap(value: unknown): value is number[][] {
```

- `unknown` - could be anything: check it
- `value is number[][]` - a **type guard**
- P prints the map as TypeScript - paste it into `ch10_array_map`

11 / 20

Try it: Tile Painter

- paint an island: 1-4 choose, 0 erases, drag to paint
- S, C, L - save, clear, load
- P, then look in the browser's console (F12)
- paste the result into `ch10_array_map/src/level.ts` and walk round it

12 / 20

A cave made in code

Making a cave: noise, smoothed, then cut-off caves filled in



- `Cave` is plain data - no Phaser. **Model**, not view
- the scene decides what it looks like

13 / 20

Two layers

```
const floor = map.createBlankLayer("floor", tileset!);
floor.weightedRandomize([
  { index: FLOOR, weight: 12 },
  { index: FLOOR_CRACKED, weight: 1 },
  { index: FLOOR_MOSSY, weight: 1 },
]);

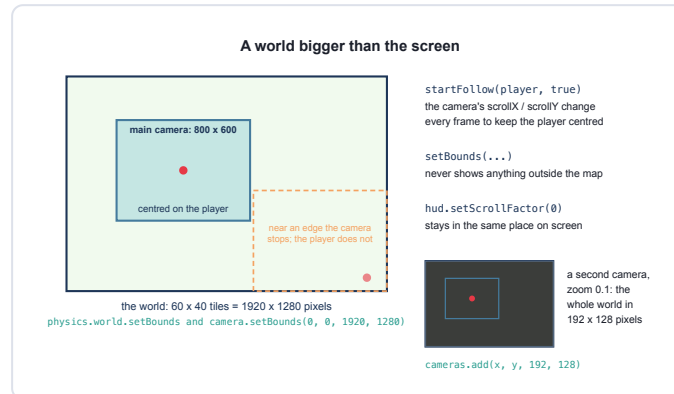
const walls = map.createBlankLayer("walls", tileset!);
walls.putTilesAt(this.wallTiles(cave), 0, 0);
walls.setCollisionByExclusion([EMPTY]); // every tile that is not empty collides
```

Walls: `-1` where the cave is open, so the floor shows through

14 / 20

The camera

```
this.physics.world.setBounds(0, 0, width, height);
this.cameras.main.setBounds(0, 0, width, height);
this.cameras.main.startFollow(this.player, true);    // true: round to whole pixels - no shimmer
this.hudText.setScrollFactor(0);                  // stays put on the screen while the camera moves
```



15 / 20

A minimap is just another camera

```
const minimap = this.cameras.add(MINI_X, MINI_Y, MINI_WIDTH, MINI_HEIGHT);
minimap.setZoom(MINI_ZOOM);
minimap.centerOn(width / 2, height / 2);

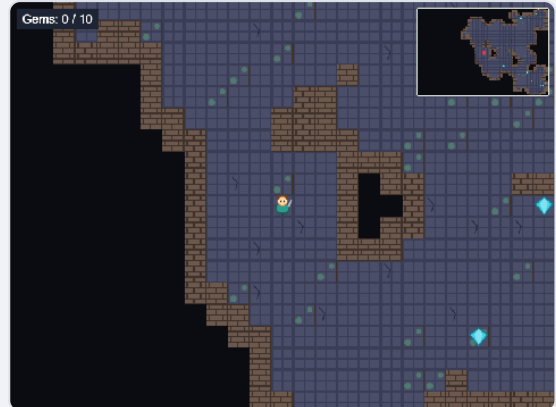
this.marker = this.add.circle(this.player.x, this.player.y, MARKER_RADIUS, 0xe63946);
this.cameras.main.ignore(this.marker);
minimap.ignore([frame, this.hudText]);
```

Every camera draws everything - unless told to `ignore` it

16 / 20

Try it: Scrolling World

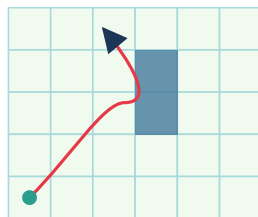
- find the ten gems; R for a new cave
- the same `Player` class as `ch10_array_map` - unchanged
- what happens without `minimap.ignore(...)` ? Without `setScrollFactor(0)` ?



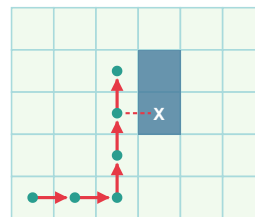
17 / 20

Free or grid movement?

Free movement and grid movement



FREE: velocity from the keys;
any pixel; the collider stops you
(this chapter's projects)



GRID: one whole tile per move, centre
to centre; look at the next tile first
(a wall or water there: do not move)

Zelda? Sokoban? Pac-Man? A roguelike?

18 / 20

Summary

- `tileset` = one picture; index from 0; `-1` = empty; `data[row][column]`
- `make.tilemap` (`data`) -> `addTilesetImage` (`picture`) -> `createLayer` (game object)
- `setCollision` + `physics.add.collider`
- `getTileAt(WorldXY)`, `worldToTileXY`, `tileToWorldXY` (top left corner)
- `putTileAt`, `removeTileAt`, `putTilesAt`, `fill`, `weightedRandomize`
- `camera.setBounds`, `startFollow`, `setScrollFactor(0)`, `cameras.add`, `ignore`

19 / 20

Challenges

1. **A new island** - moat, second bridge, maze, new start
2. **Sand slows you down** - half speed on sand
3. **Eyedropper** - right-click picks up a tile
4. **Treasure tiles** - chests in the map, collected by walking into them
5. **A map from strings** - `#`, `~`, `.`, `s`, `@`
6. **Grid movement** - one tile at a time, with tweens

Next: **Chapter 11 - Tilemaps with Tiled**

20 / 20